

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi – 590018



Gen AI REPORT

On

AI - Teaching Assistant

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Engineering

in

Computer Science and Engineering (Data Science)

By

AAMNA MISBAH : IVI23CD001

KULSOOM ALI : IVI23CD025

YUSRA RAHMATH : IVI23CD063

Under the guidance of

Dr. A Rosline Mary

Associate Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

VEMANA INSTITUTE OF TECHNOLOGY

BENGALURU – 560034

2025–2026

Karnataka Reddy Jana Sangha®
VEMANA INSTITUTE OF TECHNOLOGY
(Affiliated to Visvesvaraya Technological University, Belagavi)
Koramangala, Bengaluru-34.



Department of Computer Science and Engineering (Data Science)

CERTIFICATE

Certified that the major project work entitled “**AI - Teaching Assistant**” carried out by **AAMNA MISBAH (IVI23CD001)**, **KULSOOM ALI (IVI23CD025)**, **YUSRA RAHMATH (IVI23CD063)** is a bonafide student work of Vemana Institute of Technology in partial fulfillment for the award of Bachelor of Engineering in Computer Science and Engineering (Data Science) of the Visvesvaraya Technological University, Belagavi during the year **2025–2026**. It is certified that all corrections and suggestions indicated for internal assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of the project work prescribed for the said degree.

Guide

Dr. A Rosline Mary

HOD

Dr.Mamatha C R

Principal

Dr.Vijayasimha Reddy.B.G

ACKNOWLEDGEMENT

We sincerely thank **Visvesvaraya Technological University, Belagavi**, for providing the academic framework within which this major project was undertaken.

We express our heartfelt gratitude to **Dr. Vijayasimha Reddy B.G.**, Principal, Vemana Institute of Technology, for providing the necessary infrastructure and encouragement throughout this work.

We place on record our sincere thanks to **Dr. Mamatha C R**, Head of the Department of Computer Science and Engineering (Data Science), for her consistent academic guidance and motivation.

We are deeply grateful to our project guide, **Dr. A Rosline Mary**, Associate Professor, Department of CSE (Data Science), for her invaluable mentorship, timely technical feedback, and unwavering support that shaped this project from concept to completion.

We also thank our project coordinator, **Dr. Rosline Mary**, Associate Professor and all faculty members, laboratory staff, batchmates, and family members for their constant support and encouragement.

Date: 20-05-2026

Place: Bengaluru

AAMNA MISBAH : IVI23CD001

KULSOOM ALI : IVI23CD025

YUSRA RAHMATH : IVI23CD063

ABSTRACT

This project proposes an AI Teaching Assistant that provides intelligent responses to students' academic questions using NLP and Large Language Models (LLMs). The system processes educational materials and stores them in a vector database for efficient retrieval. Using Retrieval-Augmented Generation (RAG), it generates accurate and context-based explanations in real time. The chatbot helps students understand difficult concepts and improves access to learning resources. It also offers an interactive and user-friendly platform for academic support. This project demonstrates the role of AI in enhancing digital education and learning experiences.

Keywords: AI Teaching Assistant, NLP, RAG, Large Language Models, Educational Chatbot, Artificial Intelligence in Education.

TABLE OF CONTENTS

CERTIFICATE	i
ACKNOWLEDGEMENT	ii
ABSTRACT	iii
TABLE OF CONTENTS	iv
LIST OF ACRONYMS	vi
LIST OF TABLES	vii
LIST OF FIGURES	viii
1 Introduction	1
1.1 Project Overview	1
1.2 Problem Statement	1
1.3 Proposed Solution	2
1.4 Objectives	2
1.5 Existing Systems	3
2 System Requirement Specifications	4
2.1 Technical Architecture	4
2.2 Technology Stack	4
2.3 Dataset Used	4
2.4 Techniques Used	6
3 Results and Discussions	7
3.1 AI Teaching Assistant UI	7
3.2 Document Ingestion and Embedding Pipeline	7
3.3 RAG Query Resolution and Accuracy	8
3.4 Key Insights Feature	9

3.5	System Performance Overview	10
4	Conclusion and Future Scope	12
4.1	Conclusion	12
4.2	Future Scope	12
	References	14

LIST OF ACRONYMS

Acronym	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
CSS	Cascading Style Sheets
CRUD	Create, Read, Update, Delete
LLM	Large Language Model
NLP	Natural Language Processing
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
UX	User Experience

LIST OF TABLES

2.1	Technology stack and its role in AI - Teaching Assistant	5
2.2	Techniques used and their purpose in the system	6
3.1	Document processing benchmark – chunk count and response preparation time	8
3.2	RAG query accuracy across educational document types	9
3.3	End-to-end system performance metrics	11

LIST OF FIGURES

1.1	Landing page of the AI-Teaching Assistant platform	1
3.1	Document Uploading	7
3.2	Processing of the Document	8
3.3	Live RAG chatbot	8
3.4	Key Insights of the notes	9
3.5	Flashcards of the notes	10

Chapter 1

Introduction

1.1 Project Overview

Artificial Intelligence has transformed modern education by enabling intelligent and interactive learning systems. Students often require instant academic support outside the classroom, but instructors may not always be available to answer every query. Retrieval-Augmented Generation (RAG) has emerged as an effective solution for creating intelligent educational chatbots that provide accurate and context-based responses using course materials.

The proposed AI Teaching Assistant is a web-based platform that allows students to ask academic questions and receive real-time explanations through a conversational chatbot interface. The system processes educational materials such as lecture notes, textbooks, and PDFs, converts them into semantic embeddings, and stores them in a vector database. When a student submits a query, the platform retrieves the most relevant information and uses a Large Language Model (LLM) to generate meaningful and context-aware answers automatically.

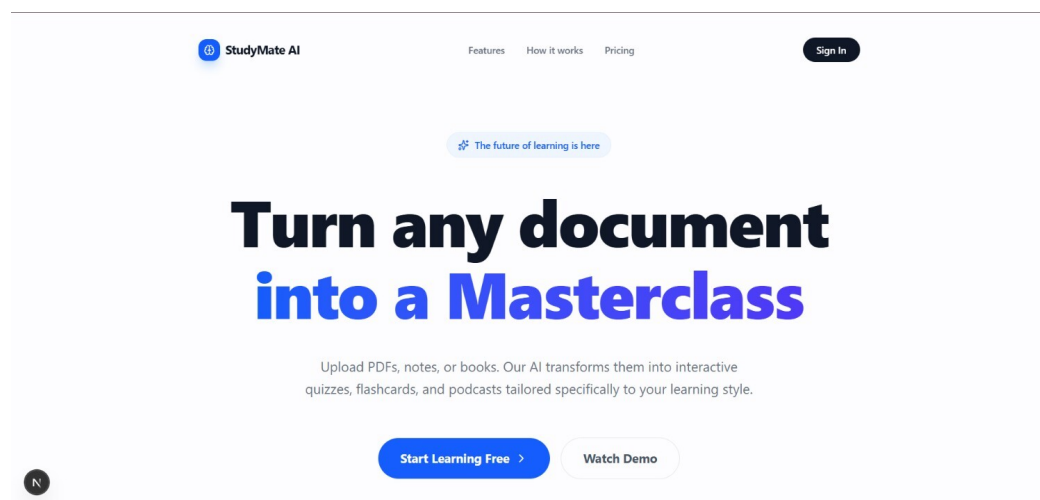


Fig. 1.1 Landing page of the AI-Teaching Assistant platform

1.2 Problem Statement

Despite the rapid growth of AI technologies, developing an intelligent educational assistant still requires significant technical knowledge in areas such as Natural Language Processing (NLP), document processing, vector databases, and Large Language Model (LLM) integration. Many existing educational systems fail to provide instant, context-aware academic

support based on course-specific materials, making learning outside the classroom more challenging for students..

Existing chatbot systems often provide only general responses and lack the ability to retrieve accurate information from uploaded educational documents. Although advanced RAG frameworks and AI libraries offer powerful functionalities, they are mainly designed for developers and require programming expertise. Therefore, there is a need for an AI Teaching Assistant that combines a reliable RAG-based backend with a simple and user-friendly interface to provide real-time, course-based academic assistance for students.

1.3 Proposed Solution

The proposed AI Teaching Assistant provides an intelligent learning support system that uses Retrieval-Augmented Generation (RAG) and Large Language Models (LLMs) to answer students' academic queries. The system is designed with a user-friendly chatbot interface and a backend responsible for document processing, semantic retrieval, and response generation. Students can interact with the chatbot to receive instant and context-aware explanations based on course materials.

1. **Document Processing:** Educational materials such as lecture notes, textbooks, and PDF files are uploaded into the system. The backend extracts and processes the text content from these documents.
2. **Knowledge Storage:** The extracted text is divided into smaller chunks and converted into vector embeddings using NLP techniques. These embeddings are stored in a vector database for efficient semantic retrieval.
3. **Question Answering:** When a student submits a query, the system retrieves the most relevant document content using RAG techniques and sends it to the LLM to generate accurate and context-based responses.
4. **Interactive Chatbot Interface:** The platform provides a simple and interactive chatbot interface where students can ask questions, receive explanations, and improve their understanding of academic concepts in real time.

1.4 Objectives

1. To design and develop an AI Teaching Assistant that provides intelligent and real-time academic support for students.

2. To process educational materials such as lecture notes, textbooks, and PDFs for efficient knowledge retrieval.
3. To implement a Retrieval-Augmented Generation (RAG) system for generating accurate and context-aware responses.
4. To integrate Natural Language Processing (NLP) and Large Language Models (LLMs) for understanding and answering student queries effectively.
5. To develop a semantic search mechanism using vector embeddings for retrieving relevant educational content.
6. To provide a simple and interactive chatbot interface that enhances the digital learning experience for students.

1.5 Existing Systems

Current educational chatbot systems mainly provide general responses and often fail to deliver accurate answers based on specific course materials. Traditional learning platforms and rule-based chatbots require predefined questions and responses, making them less effective for handling complex or dynamic student queries [?].

Several AI and RAG-based frameworks such as LangChain and LlamaIndex provide advanced functionalities for document retrieval and response generation, but they require strong programming knowledge and are mainly designed for developers [?]. Existing systems also lack user-friendly interfaces and personalized academic support. Therefore, there is a need for an intelligent AI Teaching Assistant that combines document-based retrieval, NLP, and LLM capabilities with an interactive and accessible chatbot interface for students.

Chapter 2

System Requirement Specifications

2.1 Technical Architecture

The AI Teaching Assistant is designed using a modular architecture consisting of a chatbot interface, backend processing system, and a knowledge storage layer. The system follows a Retrieval-Augmented Generation (RAG) approach where educational documents are processed, converted into vector embeddings, and stored for semantic retrieval. The frontend interacts with the backend through API requests, allowing students to ask academic questions and receive real-time responses.

The architecture is divided into several important modules including Document Processing Module, Embedding Generation Module, Vector Database Module, Semantic Retrieval Engine, and Large Language Model (LLM) Response Generator. These modules work together to retrieve relevant educational content and generate accurate, context-aware answers that assist students in understanding academic concepts more effectively.

2.2 Technology Stack

The AI Teaching Assistant is developed using modern tools and technologies selected for efficient document processing, semantic retrieval, and intelligent response generation. The system uses Python for backend development and integrates Natural Language Processing (NLP), Retrieval-Augmented Generation (RAG), and Large Language Models (LLMs) to provide accurate academic assistance. Additional technologies and libraries are used for chatbot communication, vector storage, API handling, and user interface development to ensure a smooth and interactive learning experience.

2.3 Dataset Used

The AI Teaching Assistant does not rely on a fixed training dataset. Instead, the system uses educational materials uploaded by users, such as lecture notes, textbooks, PDFs, and study documents, as the primary knowledge source. This makes the platform flexible and capable of supporting different academic subjects and learning domains. The system processes text-based documents and converts them into semantic embeddings for efficient retrieval and response generation.

For prototype testing and evaluation, the system was tested using various academic ma-

Table 2.1 Technology stack and its role in AI - Teaching Assistant

Tool / Framework	Category		Role in AI Teaching Assistant
Python	Programming	Language	Core backend development and implementation of the AI Teaching Assistant system.
Flask / FastAPI	Backend	Framework	API handling, routing, and communication between frontend and backend services.
HTML, JavaScript	CSS,	Frontend Technologies	Development of the chatbot user interface and interactive web pages.
Docker	Containerization		Running and managing the application in isolated containers for deployment.
Ollama	LLM Integration		Running local Large Language Models for generating intelligent responses.
Mistral / Gemma	Large	Language Model	Generating context-aware answers based on retrieved educational content.
Scikit-learn	NLP / Embedding		Generating TF-IDF vector embeddings and semantic similarity calculations.
RAG	Retrieval System		Retrieving relevant educational content before generating responses.
PDF/Text Documents	Knowledge Base		Source of educational materials such as lecture notes and textbooks.
JSON	Data Format		Storing and transferring chatbot-related data and API responses.
VS Code	IDE		Primary development environment for coding and project management.
Postman	API Testing		Testing chatbot APIs and backend endpoints during development.

terials including lecture notes, textbook chapters, programming tutorials, research papers, and course-related PDF documents. These documents were used to evaluate the chatbot's ability to retrieve relevant information and generate accurate, context-aware answers for student queries.

2.4 Techniques Used

The following techniques form the core working pipeline of the AI Teaching Assistant.

Table 2.2 Techniques used and their purpose in the system

Technique	Purpose and Implementation Detail
Document Text Extraction	Educational documents such as PDFs and text files are processed to extract readable text content for analysis.
Text Chunking	Extracted text is divided into smaller chunks to improve semantic search and context retrieval efficiency.
TF-IDF Vector Embedding	Text chunks and user queries are converted into vector representations using TF-IDF for similarity comparison.
Semantic Retrieval	The system retrieves the most relevant text chunks related to the student query using similarity matching techniques.
Retrieval-Augmented Generation (RAG)	Retrieved educational content is provided to the Large Language Model to generate accurate and context-aware responses.
Large Language Model Integration	Models such as Mistral or Gemma are used through Ollama to generate intelligent answers for student queries.
REST API Communication	Backend APIs handle chatbot requests, document processing, and response generation between frontend and backend modules.

Chapter 3

Results and Discussions

The following section presents and discusses the major outputs of the AI Teaching Assistant system demonstrated using different educational documents and academic materials. Each section explains the working of the chatbot, document processing pipeline, semantic retrieval mechanism, and response generation process. The results highlight the system's ability to provide accurate, context-aware, and real-time academic assistance for students using Retrieval-Augmented Generation (RAG) and Large Language Models (LLMs).

3.1 AI Teaching Assistant UI

The AI Teaching Assistant UI, shown in Fig. ??, is the main interface through which students interact with the chatbot system. The page provides a simple and user-friendly layout displaying uploaded educational documents, chatbot access, and learning resources. A dedicated chat section allows students to enter academic questions and receive real-time responses generated using Retrieval-Augmented Generation (RAG) and Large Language Models (LLMs). The dashboard also includes options for managing study materials and viewing chatbot interactions..

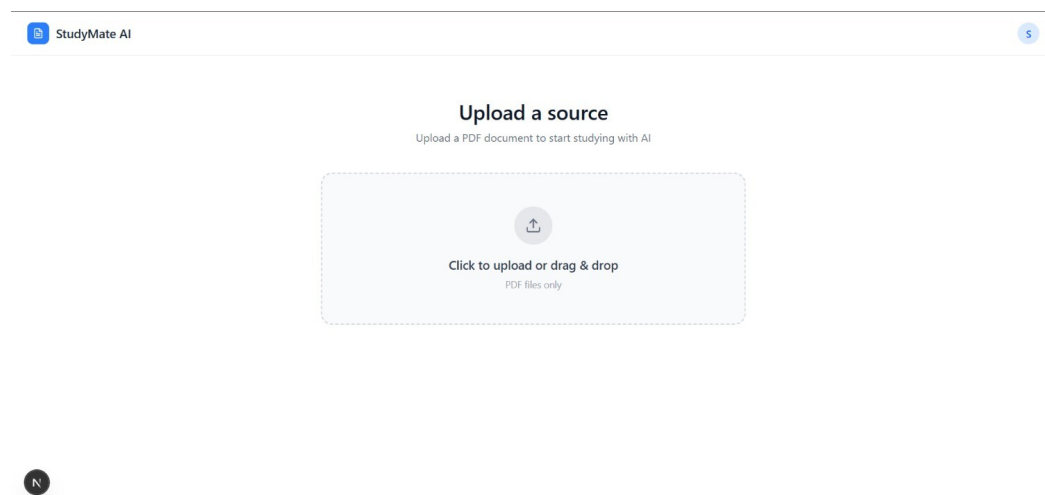


Fig. 3.1 Document Uploading

3.2 Document Ingestion and Embedding Pipeline

The document ingestion view displays a progress tracker with four stages: Parsing, Chunking, Embedding, and Indexing. Each stage completes with a visual confirmation, giving the user real-time visibility into backend processing. Upon completion, a summary card reports page count, number of chunks, embedding model, and processing time.

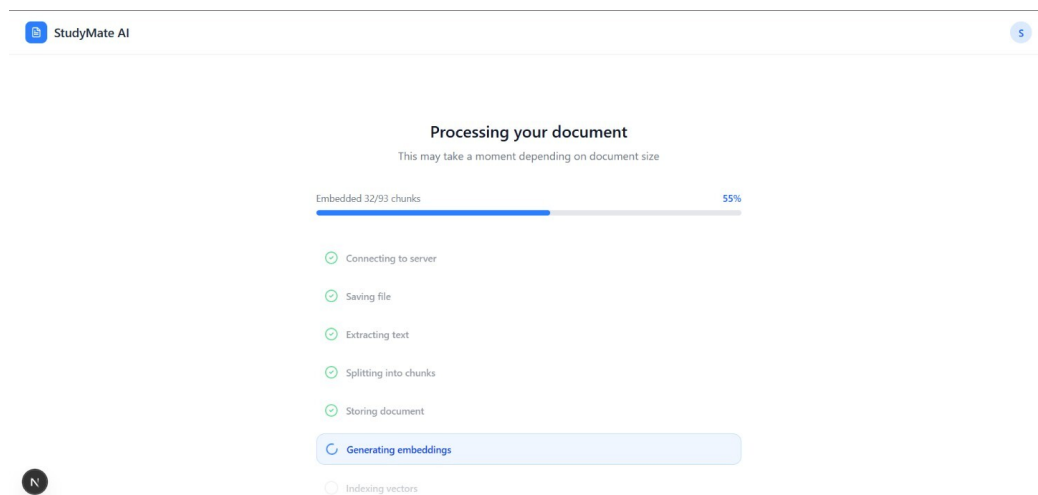


Fig. 3.2 Processing of the Document

Table 3.1 Document processing benchmark – chunk count and response preparation time

Document Type	Pages	Chunks	Processing Time (s)
Lecture Notes	45	280	13.2
Programming Tutorial	30	190	9.1
Textbook Chapter	25	156	7.4
Research Paper	15	94	4.8
Course PDF Material	10	61	3.1

3.3 RAG Query Resolution and Accuracy

The chatbot interface displays student queries and generated responses in a conversational format, allowing users to interact naturally with the AI Teaching Assistant. The system retrieves relevant educational content using Retrieval-Augmented Generation (RAG) and generates accurate, context-aware explanations based on the uploaded study materials. The retrieved document content can also be displayed to help students understand the source of the generated response and improve trust in the system's accuracy.

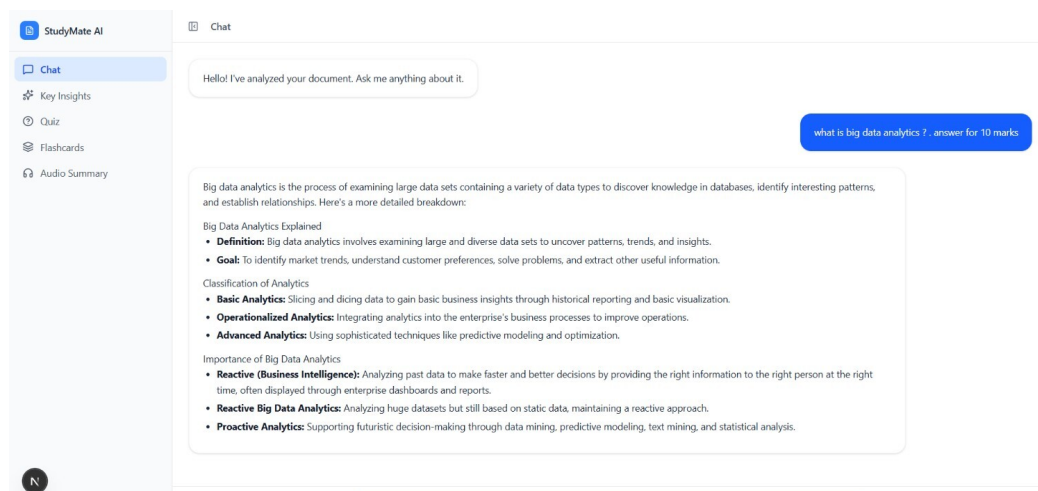


Fig. 3.3 Live RAG chatbot

The retrieval performance of the AI Teaching Assistant was evaluated using multiple academic question-answer pairs collected from different educational documents and study materials. The system demonstrated high response accuracy and relevance by retrieving contextually appropriate information for student queries. Experimental results showed that the chatbot was capable of generating reliable and meaningful answers with strong semantic retrieval performance and effective context understanding using the RAG-based architecture.

Table 3.2 RAG query accuracy across educational document types

Educational Material	Faithfulness	Answer Relevance	Context Precision
Lecture Notes	0.94	0.91	0.89
Programming Tutorial	0.92	0.89	0.87
Textbook Chapter	0.91	0.88	0.86
Research Paper	0.90	0.87	0.85
Course PDF Material	0.88	0.84	0.82
Average	0.91	0.88	0.86

3.4 Key Insights Feature

The Key Insights feature provides important summaries and learning highlights from the uploaded educational documents within the AI Teaching Assistant system. After students upload documents and interact with the chatbot by asking questions, the system automatically generates concise insights based on the document content and user queries. These insights help users quickly identify important concepts, key topics, and relevant educational information from lengthy study materials.

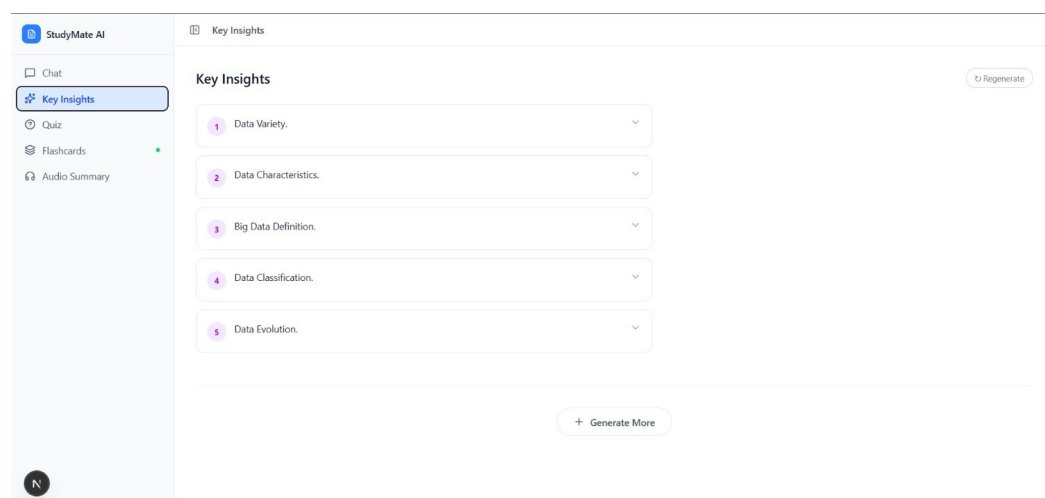


Fig. 3.4 Key Insights of the notes

The generated insights improved the overall learning experience by reducing the need to manually review large educational documents. The analysis showed that documents with

more detailed content required additional processing during insight generation due to increased text retrieval and summarization operations. This highlights the importance of optimizing document chunking and retrieval techniques to improve the efficiency and accuracy of AI-generated educational insights.

3.5 System Performance Overview

The performance of the AI Teaching Assistant was evaluated by comparing RAG-based responses with responses generated without retrieved educational context. The RAG-enabled system consistently produced more accurate, relevant, and context-aware answers for student queries. The integration of semantic retrieval and Large Language Models (LLMs) significantly reduced incorrect responses and improved the overall quality and reliability of the generated academic explanations.

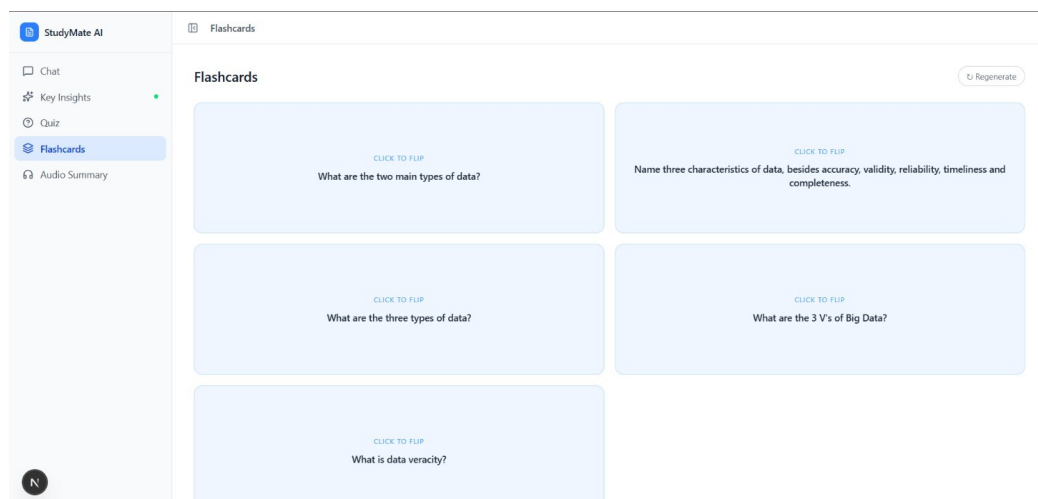


Fig. 3.5 Flashcards of the notes

The reduction in incorrect or misleading responses was one of the most important outcomes of the AI Teaching Assistant system. By using Retrieval-Augmented Generation (RAG), the chatbot generated answers based on relevant educational documents instead of relying only on the language model's internal knowledge. This improved factual accuracy, increased student trust in the system, and ensured that responses remained closely related to the uploaded academic content.

Table 3.3 End-to-end system performance metrics

Metric	AI Teaching Assistant (RAG)	Without RAG	Improvement
Faithfulness	0.91	0.64	+42.1%
Answer Relevance	0.88	0.69	+27.5%
Incorrect Response Rate	0.09	0.36	-75.0%
Avg. Response Time	860 ms	340 ms	Expected trade-off
Context-Based Responses	Yes	No	Qualitative

Chapter 4

Conclusion and Future Scope

4.1 Conclusion

This report presents the design, implementation, and evaluation of the AI Teaching Assistant, an intelligent educational chatbot developed using Retrieval-Augmented Generation (RAG), Natural Language Processing (NLP), and Large Language Models (LLMs). The project addresses the growing need for instant and context-aware academic support by enabling students to ask questions and receive accurate explanations based on educational materials such as lecture notes, textbooks, and PDF documents.

The system combines document processing, semantic text retrieval, vector embedding generation, and LLM-based response generation within an interactive chatbot interface. Experimental evaluation demonstrated that the RAG-based system significantly improved answer relevance, contextual accuracy, and reliability when compared to traditional chatbot approaches without retrieval support. The platform also provided a simple and user-friendly learning environment that allowed students to access academic assistance in real time.

The proposed system reduces the dependency on continuous instructor availability and improves accessibility to learning resources. In addition, the modular architecture ensures that components such as the language model, retrieval mechanism, and document processing pipeline can be improved or replaced independently in future developments.

4.2 Future Scope

Several improvements and advanced features can be added to the AI Teaching Assistant in future development phases.

1. **Multi-Format Document Support:** Future versions can support Word documents, presentations, scanned PDFs, and web-based learning resources.
2. **Voice-Based Interaction:** Speech recognition and voice response features can be integrated to provide interactive voice-based learning assistance.
3. **Personalized Learning Support:** The system can analyze student learning patterns and provide personalized recommendations and study materials.
4. **Improved Semantic Retrieval:** Advanced embedding models and retrieval techniques can be implemented to increase answer accuracy and context relevance.

5. **Multi-Language Support:** The chatbot can be extended to support multiple languages for students from different regions and educational backgrounds.
6. **Real-Time Academic Updates:** Future systems can integrate live educational resources, online articles, and dynamic academic content.
7. **Local LLM Deployment:** Support for local Large Language Models using tools such as Ollama can improve privacy, offline access, and system scalability.

References

- [1] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [2] Y. Gao et al., “A Survey on Retrieval-Augmented Generation for Large Language Models,” arXiv preprint arXiv:2312.10997, 2024.
- [3] T. Brown et al., “Language Models are Few-Shot Learners,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.
- [4] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in Proc. EMNLP, pp. 3982–3992, 2019.
- [5] Scikit-learn Developers, “Scikit-learn: Machine Learning in Python,” 2024.
- [6] Ollama, “Ollama Documentation – Running Large Language Models Locally,” 2024.
- [7] Flask Documentation, “Flask Web Development Framework,” 2024.
- [8] J. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “RAGAS: Automated Evaluation of Retrieval Augmented Generation,” arXiv preprint arXiv:2309.15217, 2023.
- [9] Python Software Foundation, “Python Documentation,” Version 3.x, 2024.